

Advanced Programming

Project assignment – 5th stage

Model observability

Logs

Stage 5

- All students must update the project and *checkout* the `main` *branch*
- The student responsible for this task should start by creating a new *branch* from the `main` *branch*
 - *Branch* name: **stage5**
 - The student who creates the *branch* should *push* it to *GitHub* to make it available to the rest of the group
- The remaining students should update the project and then *checkout* the **stage5** *branch*

Stage 5

- The deadline to perform the *Pull Request* and the *Squash and Merge* is 23:59 on the day before the lesson during the week of May 12-16
 - Worker-students should complete this stage by 2025.05.17
- The *Squash and Merge* into the *main* branch must be performed on the *GitHub* website
 - The *branch* must not be deleted after the *Merge* is completed

Stage 5

- In the basis presented for the implementation of MVC, it is understood that, every time there is a change to the data, a call to the `update` method must be made so that the view is updated
- The integration of mechanisms for automatic view updates will simplify this process

Stage 5

- Modify the application so that updates to the graphical interface are performed 'automatically' by using the functionalities provided by *property changes*, as studied in the theoretical class
- Update the class that manages access to the data model (`ChessGameManager`) so that it triggers appropriate property changes to signal modifications and relevant events throughout the application
 - A second window (`Stage`), similar to the main one, can be configured to verify that updates via property changes are working correctly

Stage 5

- Create an instance of `PropertyChangeSupport` in the `ChessGameManager` class
- This object is responsible for managing the *Publisher/Subscriber* pattern, supervising the topics of interest and their subscribers (*listeners*)
 - It can also be used to implement the *Observer/Observable* pattern, allowing the management of global interests rather than interest in specific topics
- The constructor of `PropertyChangeSupport` takes the *Observable* object as a parameter, which acts as the source of the events being generated

Stage 5

- A method `addPropertyChangeListener` should be added to the `ChessGameManager` class to allow *observers/subscribers* to register their interest in events that report changes to the data model
- All methods that modify the data model should generate an event using the `firePropertyChange` function
- Consider defining distinct topics of interest (i.e., different property names) to signal more specific types of changes
 - For example, *"boardState"* for changes in the board configuration, or *"currentPlayer"* to indicate a change in the active player
 - This allows listeners to react only to the events they are interested in

Stage 5

- Add a `ModelLog` class to report logs generated by the data model, such as board creation, moves, thrown exceptions, or other errors
- The class must follow the *Singleton* pattern and provide appropriate methods to manage the logs: adding new entries, retrieving existing logs, and clearing all logs
- This logging class must also be observable via its own `PropertyChangeSupport` instance, independent from the one used in the `ChessGameManager` class
- A new `Stage` should be added to the application at startup, allowing the user to view the generated logs
 - This view should use a *JavaFX* `ListView` to display the log entries
 - A button or another suitable UI control should be included to allow clearing all logs